

Skill 10 os

SESSION 19

Task 1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s10.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int sig) {  
    printf("Signal %d received and handled.\n", sig);  
}  
  
int main() {  
    signal(SIGINT, handler);  
    printf("Signal handler registered. Press Ctrl+C...\n");  
    sleep(10);  
    printf("Program completed safely.\n");  
    return 0;  
}
```

Output:

```
Signal handler registered. Press Ctrl+C...  
^CSignal 2 received and handled.  
Program completed safely.  
rohita@DESKTOP-1UHU58C:~$
```

Observation: The signal handler successfully handled SIGINT generated by **Ctrl+C**, allowing the program to continue execution without terminating unexpectedly.

Task 2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

Code:

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s10T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = -1;

void sigint_handler(int sig) {
    printf("\nShell: SIGINT received.\n");
    if (child_pid > 0) {
        printf("Shell: Forwarding SIGINT to foreground job...\n");
        kill(child_pid, SIGINT);
    }
}

int main() {
    int status;
    signal(SIGINT, sigint_handler);

    printf("Shell PID: %d\n", getpid());
    printf("Creating foreground job...\n");

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        exit(1);
    }

    if (child_pid == 0) {
        signal(SIGINT, SIG_DFL);
        printf("Foreground Job PID: %d\n", getpid());
        printf("Job is running... Press Ctrl+C to terminate it.\n");

        while (1) {
            printf("Foreground job working...\n");
            sleep(2);
        }
    }
    else {
        printf("Shell: Waiting for foreground job...\n");

        waitpid(child_pid, &status, 0);

```

```

        waitpid(child_pid, &status, 0);

        if (WIFSIGNALED(status))
            printf("Shell: Foreground job terminated by signal %d.\n",
                WTERMSIG(status));

        child_pid = -1;

        printf("Shell: Job state updated to TERMINATED.\n");
        printf("Shell: Ready for next command.\n");
    }

    return 0;
}

```

Output:

```
rohita@DESKTOP-1UHU58C:~$ ./a.out
Shell PID: 9375
Creating foreground job...
Shell: Waiting for foreground job...
Foreground Job PID: 9376
Job is running... Press Ctrl+C to terminate it.
Foreground job working...
Foreground job working...
Foreground job working...
Foreground job working...
^C
Shell: SIGINT received.
Shell: Forwarding SIGINT to foreground job...
Shell: Foreground job terminated by signal 2.
Shell: Job state updated to TERMINATED.
Shell: Ready for next command.
```

Observation: The program successfully captures the SIGINT signal generated by **Ctrl+C**, protects the shell process, and forwards the signal to the foreground job. The foreground process is terminated and its job state is updated accordingly.

SESSION 20

Task 1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Code

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s10T
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

pid_t child_pid=-1;
int job_state=0;

void sigtstp_handler(int sig){
    if(child_pid>0){
        printf("\nShell: SIGTSTP received (Ctrl+Z).\n");
        printf("Shell: Suspending foreground job...\n");
        kill(child_pid,SIGSTOP);
        job_state=1;
    }
}

void resume_job(){
    if(child_pid>0 && job_state==1){
        printf("Shell: Resuming suspended job...\n");
        kill(child_pid,SIGCONT);
        job_state=0;
        printf("Shell: Job resumed.\n");
    }else{
        printf("No suspended job found.\n");
    }
}

int main(){
    char command[50];
    int status;

    signal(SIGTSTP,sigtstp_handler);

    printf("=====\n");
    printf("      JOB SUSPENSION MANAGER\n");
    printf("=====\n");
    printf("Commands: start, resume, exit\n");

    while(1){
        printf("\n$ ");
        fflush(stdout);

```

```

fflush(stdout);

fgets(command, sizeof(command), stdin);
command[strcspn(command, "\n")] = '\0';

if(strcmp(command, "exit") == 0)
    break;

if(strcmp(command, "resume") == 0){
    resume_job();
    continue;
}

if(strcmp(command, "start") == 0){
    if(child_pid > 0 && job_state == 0){
        printf("A job is already running.\n");
        continue;
    }

    child_pid = fork();

    if(child_pid < 0){
        perror("fork");
        exit(1);
    }

    if(child_pid == 0){
        signal(SIGTSTP, SIG_DFL);
        printf("Foreground job started. PID: %d\n", getpid());

        while(1){
            printf("Job is running...\n");
            sleep(2);
        }
    }
    else{
        waitpid(child_pid, &status, WUNTRACED);

        if(WIFSTOPPED(status)){
            job_state = 1;
            printf("\nJob Table: Job is STOPPED.\n");
            printf("PID %d state preserved.\n", child_pid);
            printf("Use 'resume' to continue the job.\n");
        }
    }
}

```

```

        if(WIFEXITED(status)){
            job_state = 2;
            printf("Job completed.\n");
        }
    }
    else{
        printf("Use: start, resume, or exit\n");
    }
}

if(child_pid > 0 && job_state == 0)
    kill(child_pid, SIGTERM);

printf("Exiting job manager.\n");
return 0;
}

```

Output


```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>
int main(){
    pid_t pid,pgid;
    int status;
    printf("=====\n");
    printf("    PROCESS GROUP & TERMINAL TEST\n");
    printf("=====\n");
    signal(SIGTTOU,SIG_IGN);
    signal(SIGTTIN,SIG_IGN);
    pid=fork();
    if(pid<0){
        perror("fork");
        exit(1);
    }
    if(pid==0){
        setpgid(0,0);
        pgid=getpgrp();
        signal(SIGINT,SIG_DFL);
        signal(SIGTERM,SIG_DFL);
        printf("\nChild PID: %d\n",getpid());
        printf("Child Process Group ID: %d\n",pgid);
        printf("Child is running in foreground.\n");
        printf("Press Ctrl+C to send SIGINT to the foreground group.\n");
        for(int i=1;i<=10;i++){
            printf("Child working... %d\n",i);
            sleep(2);
        }
        printf("Child completed successfully.\n");
        exit(0);
    }else{
        setpgid(pid,pid);
        pgid=pid;
        printf("\nParent PID: %d\n",getpid());
        printf("Created Child PID: %d\n",pid);
        printf("Assigned Process Group ID: %d\n",pgid);
        printf("Transferring terminal control to child group...\n");
        if(tcsetpgrp(STDIN_FILENO,pgid)==-1)
            perror("tcsetpgrp");
        printf("Terminal control transferred.\n");
        if(waitpid(pid,&status,WUNTRACED)==-1)

```

```

            printf("Terminal control transferred.\n");
            if(waitpid(pid,&status,WUNTRACED)==-1)
                perror("waitpid");
            printf("\nChild process finished or stopped.\n");
            printf("Restoring terminal control to parent...\n");
            if(tcsetpgrp(STDIN_FILENO,getpgrp())==-1)
                perror("tcsetpgrp");
            printf("Terminal control restored to parent.\n");
            if(WIFEXITED(status))
                printf("Job State: COMPLETED\n");
            else if(WIFSIGNALED(status))
                printf("Job State: TERMINATED by signal %d\n",WTERMSIG(status));
            else if(WIFSTOPPED(status))
                printf("Job State: STOPPED by signal %d\n",WSTOPSIG(status));
        }
        return 0;
    }
}

```

Output:

```
=====
PROCESS GROUP & TERMINAL TEST
=====

Parent PID: 9497
Created Child PID: 9498
Assigned Process Group ID: 9498
Transferring terminal control to child group...
Terminal control transferred.

Child PID: 9498
Child Process Group ID: 9498
Child is running in foreground.
Press Ctrl+C to send SIGINT to the foreground group.
Child working... 1
Child working... 2
Child working... 3
Child working... 4
Child working... 5
Child working... 6
Child working... 7
Child working... 8
Child working... 9
Child working... 10
Child completed successfully.

Child process finished or stopped.
Restoring terminal control to parent...
Terminal control restored to parent.
Job State: COMPLETED
```

Observation: The program successfully creates process groups, transfers terminal control, handles signals, and restores terminal control to the parent process.